

SIMATS ENGINEERING (Saveetha School of Engineering)

Department of Computer Science and Engineering

ARTIFICIAL INTELLIGENCE LAB REPORT

Assessment Tool 2 – Constraint-Based Problem Solving

Course Name: Artificial Intelligence

Course Code: CSA1704

Name: MITHRA DEVI S

Register Number: 192524203

Department: ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Title

Constraint-Based Problem Solving using Backtracking Search, Breadth-First Search (BFS), and Uniform Cost Search (UCS)

Aim

To implement Constraint-Based Problem Solving techniques using Python by solving the Hospital Doctor Scheduling problem with Backtracking Search, Robot Grid Navigation using Breadth-First Search (BFS), and Rescue Robot Navigation using Uniform Cost Search (UCS).

Objectives

- To understand the concept of Constraint Satisfaction Problems (CSP).
- To implement the Backtracking Search algorithm for scheduling problems.
- To implement the Breadth-First Search (BFS) algorithm for path finding.
- To implement the Uniform Cost Search (UCS) algorithm for least-cost navigation.
- To analyze intelligent agent behavior in constrained environments.

Problem Statement

This experiment consists of three Artificial Intelligence problems:

1. Assign doctors to hospital shifts while satisfying all scheduling constraints using **Backtracking Search**.
 2. Find the shortest path for a robot in a 5×5 grid with obstacles using **Breadth-First Search (BFS)**.
 3. Navigate an autonomous rescue robot through a disaster-affected environment with risky zones using **Uniform Cost Search (UCS)** to minimize total travel cost.
-

Software Requirements

- Python 3.x
 - IDLE / VS Code / PyCharm
 - Windows 10/11
-

Theory

Constraint-Based Problem Solving is an important area of Artificial Intelligence where solutions must satisfy a given set of constraints. AI search algorithms explore possible states until a valid or optimal solution is found.

Backtracking Search

Backtracking is a Depth-First Search (DFS)-based algorithm used for solving Constraint Satisfaction Problems (CSP). It assigns values step by step and backtracks whenever a constraint is violated.

Breadth-First Search (BFS)

Breadth-First Search explores all neighboring nodes before moving to the next level. It guarantees the shortest path in an unweighted graph.

Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest cumulative path cost first. It guarantees the least-cost solution in weighted graphs where edge costs may vary.

Algorithm

Algorithm 1: Hospital Doctor Scheduling (Backtracking Search)

1. Define doctors and available shifts.
 2. Assign one shift to each doctor.
 3. Check all scheduling constraints.
 4. If a constraint is violated, backtrack.
 5. Continue until a valid assignment is found.
 6. Display the final doctor schedule.
-

Algorithm 2: Robot Grid Navigation (BFS)

1. Create the 5×5 grid.
 2. Mark the Start (S), Goal (G), and obstacles.
 3. Insert the start node into the queue.
 4. Expand neighboring nodes level by level.
 5. Ignore blocked cells.
 6. Continue until the goal is reached.
 7. Display the shortest path and total cost.
-

Algorithm 3: Rescue Robot Navigation (Uniform Cost Search)

1. Create the disaster environment grid.
2. Mark risky zones and blocked cells.
3. Insert the start node into the priority queue.

4. Expand the node with the lowest cumulative cost.
 5. Add additional cost for risky zones.
 6. Continue until the goal node is reached.
 7. Display the least-cost path and total cost.
-

Program

Write the combined menu-driven Python program here.

Sample Output

Main Menu

===== AI ASSESSMENT TOOL 2 =====

1. Hospital Doctor Scheduling (Backtracking)
 2. Robot Grid Navigation (BFS)
 3. Rescue Robot (Uniform Cost Search)
 4. Exit
-

Output 1 – Doctor Scheduling

Enter your choice: 1

Valid Schedule

D1 -> Morning

D2 -> Afternoon

D3 -> Night

Output 2 – Robot Grid Navigation

Enter your choice: 2

Goal Reached

Path :

[(0,0), (0,1), (0,2), (1,2), (2,2),
(2,3), (2,4), (3,4), (4,4)]

Cost : 8

Output 3 – Rescue Robot

Enter your choice: 3

Goal Reached

Optimal Path :

[(0,0), (0,1), (0,2), (1,2), (2,2),
(2,3), (2,4), (3,4), (4,4)]

Total Cost : 8

Output 4 – Exit

Enter your choice: 4

Thank You

Applications

- Hospital staff scheduling
 - Employee duty allocation
 - Robot path planning
 - Autonomous rescue systems
 - Warehouse automation
 - Logistics and transportation
 - Route optimization
 - Smart navigation systems
-

Advantages

- Finds valid solutions while satisfying all constraints.
 - Efficiently solves scheduling and pathfinding problems.
 - Guarantees optimal solutions in BFS and UCS.
 - Demonstrates important AI search techniques.
 - Easy to understand and implement using Python.
-

Result

The three Artificial Intelligence problems were successfully implemented using Python. Backtracking Search generated a valid doctor schedule while satisfying all constraints. Breadth-First Search found the shortest path for robot navigation in the grid environment.

Uniform Cost Search successfully determined the least-cost path for the rescue robot by considering both movement costs and risky zones. The program executed successfully and produced the expected results.

Conclusion

This experiment demonstrated the practical application of **Constraint-Based Problem Solving** in Artificial Intelligence. The **Backtracking Search** algorithm effectively solved the doctor scheduling problem by satisfying all constraints. The **Breadth-First Search (BFS)** algorithm successfully identified the shortest path for robot navigation in an unweighted grid, while the **Uniform Cost Search (UCS)** algorithm determined the least-cost path in a weighted environment with risky zones. These algorithms are widely used in real-world applications such as scheduling, robotics, navigation, logistics, and autonomous systems, making them fundamental techniques in Artificial Intelligence.